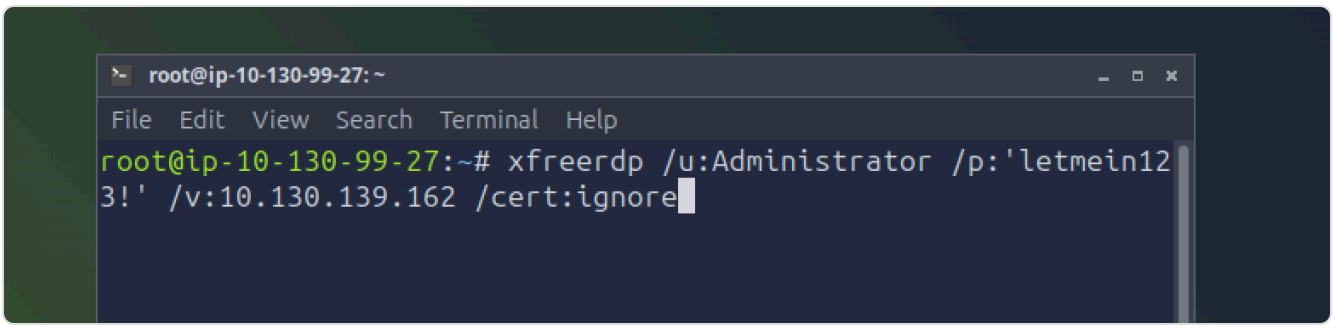
Writeup - Investigating Windows (TryHackMe)

Reconstruir un ataque a partir de un log que casi se borra solo

Esta sala la hice como parte de la Práctica 2 de Seguridad en los Sistemas Informáticos (auditoría y análisis forense). No hay explotación: TryHackMe entrega una máquina Windows Server **ya comprometida** y el ejercicio consiste en ponerse en el papel del analista forense que llega después del incidente — reconstruir qué hizo el atacante, cuándo, con qué cuentas y qué dejó de persistencia, a partir únicamente de lo que queda en el sistema.

Acceso por RDP desde el AttackBox de la sala, con las credenciales que entrega el enunciado:

```
xfreerdp /u:Administrator /p:'letmein123!' /v:10.130.139.162 /cert:ignore
```



Conectando por RDP desde el AttackBox hacia la máquina objetivo

Reconocimiento

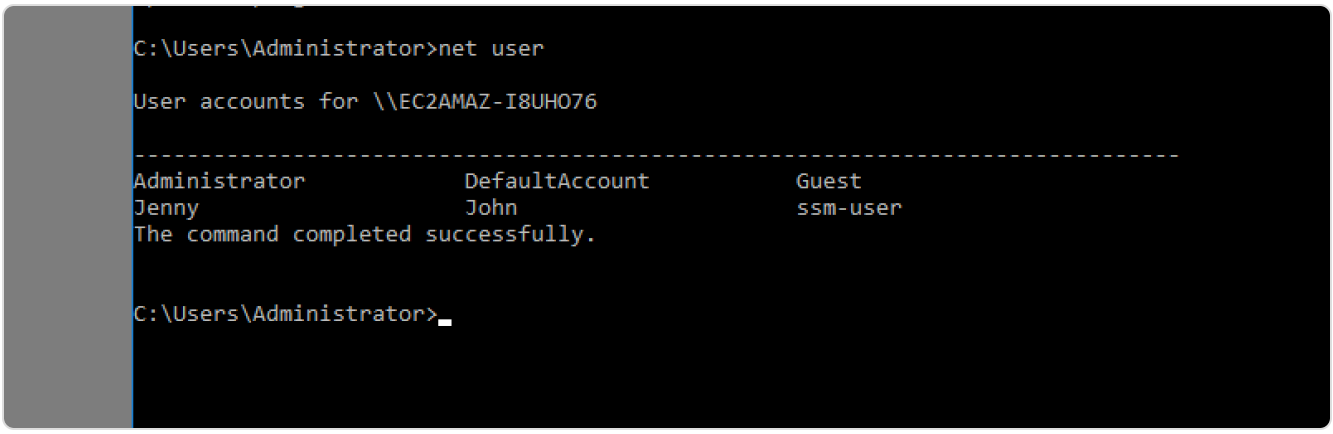
Nada más entrar al escritorio remoto, antes de tocar nada, ya había una ventana abierta: una sesión de **PsExec** (`C:\TMP\p.exe`) congelada intentando conectar a `10.34.2.3`. No la ejecuté yo — es un rastro que había dejado el propio atacante, visible desde el primer segundo. Con eso ya tenía dos datos gratis antes de investigar nada: el atacante usó PsExec para movimiento lateral/ejecución remota, y `C:\TMP` es su carpeta de trabajo en esta máquina.

Con `winver` confirmé la versión del sistema: Windows Server 2016.

Quién es quién: usuarios y privilegios

Lo primero, mapear las cuentas locales:

```
net user
```



`net user` — cuentas locales de la máquina

De las seis cuentas que salen, `ssm-user` no tiene nada que ver con el incidente — es la cuenta de servicio que AWS Systems Manager añade automáticamente en cualquier instancia EC2 (la máquina se llama `EC2AMAZ-I8UHO76`, así que tiene sentido). Es ruido de la plataforma, no del ataque, y conviene descartarlo pronto para no perder tiempo investigándolo.

Para saber cuándo inició sesión cada usuario probé primero tirar del log de Seguridad con el Visor de Eventos, filtrando por el evento 4624 (inicio de sesión exitoso) — y no encontré nada, ni un solo resultado (la explicación de por qué viene más abajo, es la parte interesante del ejercicio). Así que cambié de estrategia y usé el campo "Last logon" que Windows guarda por cuenta, sin depender del log en vivo:

```
net user John
```

```
C:\Users\Administrator>net user John
User name                    John
Full Name                    John
Comment
User's comment
Country/region code          000 (System Default)
Account active               Yes
Account expires              Never

Password last set            3/2/2019 5:48:19 PM
Password expires             Never
Password changeable          3/2/2019 5:48:19 PM
Password required            Yes
User may change password     Yes

Workstations allowed         All
Logon script
User profile
Home directory
Last logon                   3/2/2019 5:48:32 PM

Logon hours allowed          All

Local Group Memberships      *Users
Global Group memberships     *None
The command completed successfully.
```

`net user John` — último inicio de sesión: 3/2/2019 5:48:32 PM

Después miré qué cuentas tenían privilegios de administrador aparte de la propia `Administrator`:

```
net localgroup administrators
```

```
C:\Users\Administrator>net localgroup administrators
Alias name     administrators
Comment        Administrators have complete and unrestricted access to the computer/domain

Members

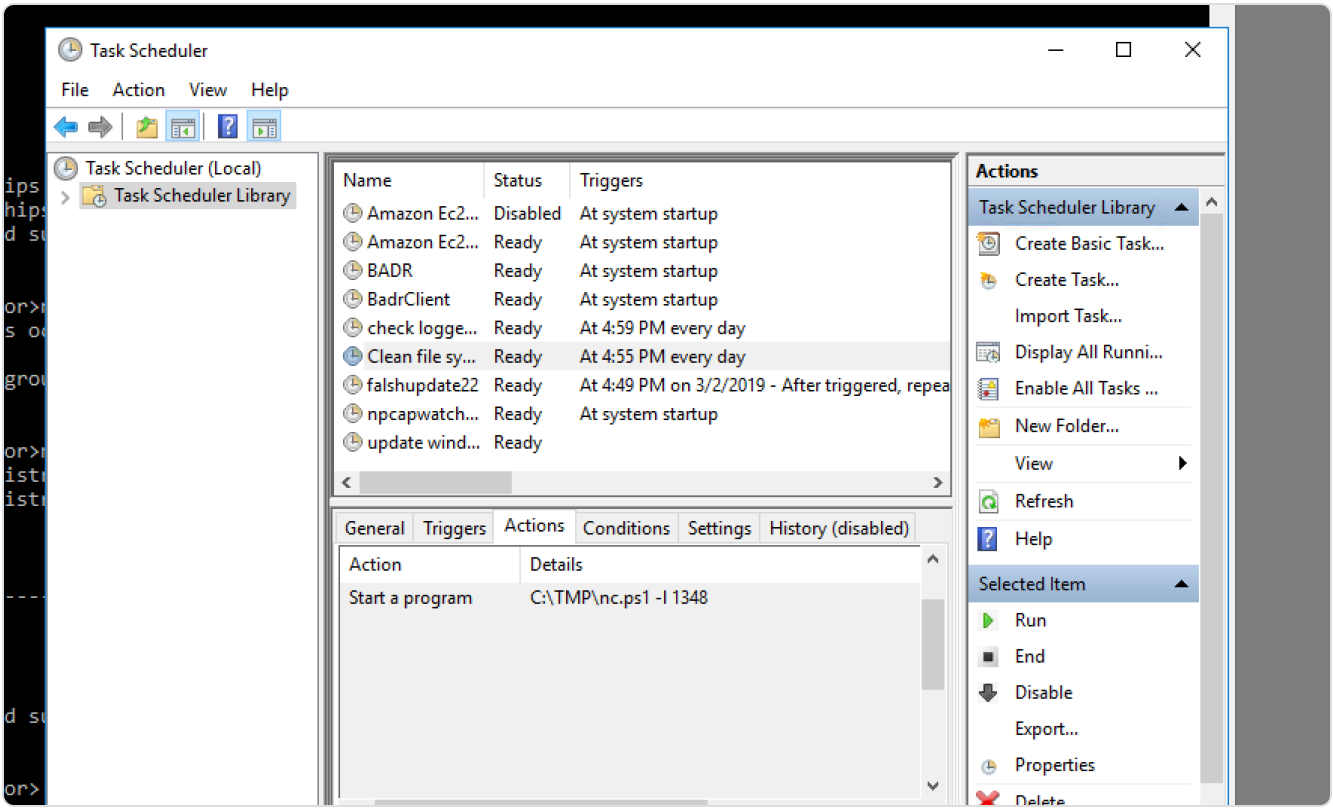
-------------------------------------------------------------------------------
Administrator
Guest
Jenny
ssm-user
The command completed successfully.
```

`net localgroup administrators`

De las cuatro cuentas en el grupo, otra vez `ssm-user` es ruido de AWS. Las dos que de verdad importan son **Guest** y **Jenny** — una cuenta por defecto a la que alguien le dio privilegios que no debería tener, y una cuenta que no estaba en el `net user` original con ese nombre esperado para un empleado normal. Ambas son el tipo de escalada silenciosa que un atacante deja para no depender de la cuenta de Administrator en próximas visitas.

La tarea que no limpia nada

Para la persistencia, revisé el Task Scheduler (`taskschd.msc`) buscando algo que no encajara entre las tareas legítimas del sistema (las de Amazon EC2, BADR, etc.):



Task Scheduler Library — la tarea "Clean file system" apuntando a C:\TMP\nc.ps1

La tarea **"Clean file system"** tiene justo el nombre que uno ignoraría en una lista larga — suena a mantenimiento rutinario. Se dispara **todos los días a las 4:55 PM**, y en su pestaña Actions ejecuta:

```
C:\TMP\nc.ps1 -l 1348
```

`nc.ps1` es una implementación de Netcat en PowerShell, y `-l 1348` la pone en modo escucha en el puerto 1348. Es decir: una puerta trasera que se relanza sola cada día a la misma hora, en la misma carpeta `C:\TMP` donde ya habíamos visto el PsExec al principio — mismo cuartel general del atacante.

Cuándo ocurrió todo: reconstruir la fecha por timestamps NTFS

Con el log de Seguridad inutilizable para esto (otra vez, ver más abajo), até la fecha del compromiso a la evidencia que sí seguía intacta: los timestamps del propio `nc.ps1` en disco.

```
Get-Item "C:\TMP\nc.ps1"
```

```
PS C:\Users\Administrator> Get-Item "C:\TMP\nc.ps1"


    Directory: C:\TMP


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         3/2/2019   4:37 PM          37640 nc.ps1
```

Primer vistazo: solo sale LastWriteTime por defecto

Esto solo me dio el `LastWriteTime`. Para comparar ambas fechas a la vez pedí explícitamente los dos campos:

```
Get-Item "C:\TMP\nc.ps1" | Select-Object CreationTime, LastWriteTime
```

```
PS C:\Users\Administrator> Get-Item "C:\TMP\nc.ps1" | Select-Object CreationTime, LastWriteTime

CreationTime        LastWriteTime
------------        -------------
3/2/2019 4:45:03 PM 3/2/2019 4:37:37 PM


PS C:\Users\Administrator> _
```

CreationTime: 3/2/2019 4:45:03 PM — LastWriteTime: 3/2/2019 4:37:37 PM

Lo interesante es que **LastWriteTime es anterior a CreationTime**. Esa es la firma típica de un archivo copiado: NTFS conserva la fecha de modificación del fichero original (4:37 PM, cuando se escribió en la máquina del atacante) pero marca como "creación" el momento en que aterrizó en este disco en concreto (4:45 PM, al copiarlo). Encaja además con otra tarea que vi en el Task Scheduler, `falshupdate22`, programada para dispararse por primera vez a las **4:49 PM del 3/2/2019** — cuatro minutos después de dejar caer el backdoor. Con eso, fecha de compromiso: **2 de marzo de 2019**, sobre las 16:37–16:49.

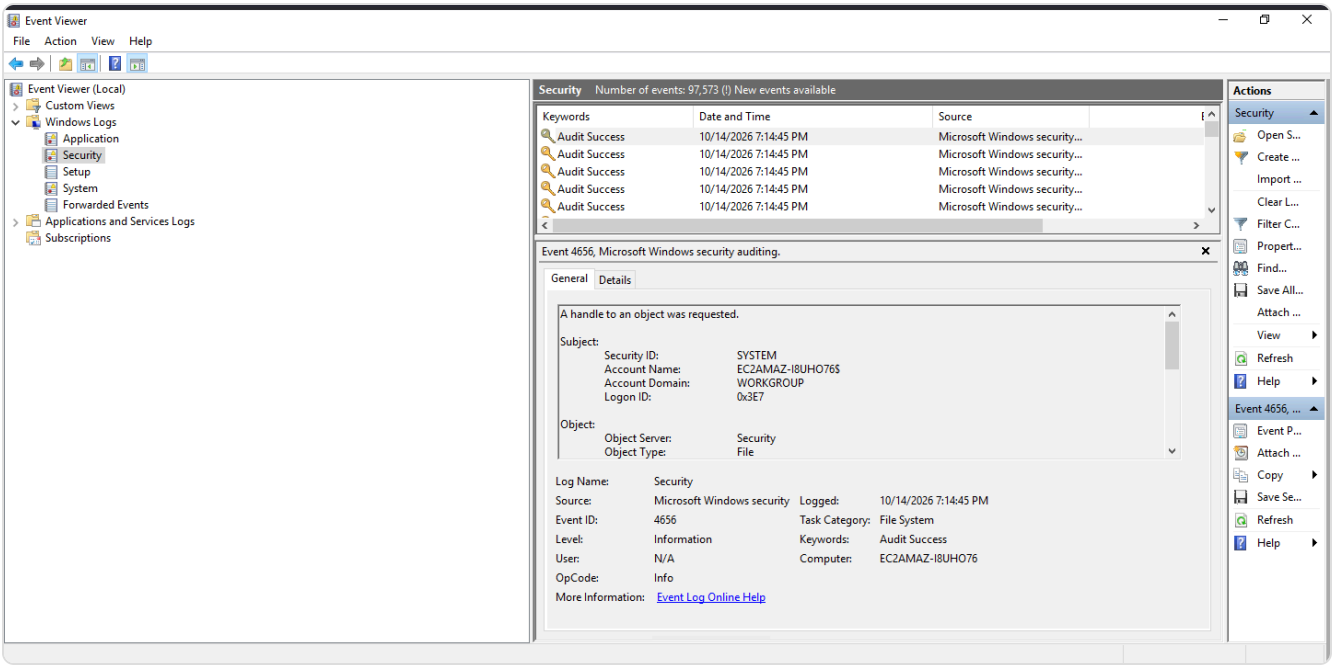
Lo que no sale en la mayoría de writeups

La sala también pregunta a qué hora Windows asignó por primera vez privilegios especiales a un logon nuevo — eso corresponde al evento **4672** ("Special privileges assigned to new

logon") en el log de Seguridad. Lo filtré por Event Viewer y también por PowerShell, y en ambos sitios, nada:

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4672} -MaxEvents 5
# Get-WinEvent : No events were found that match the specified selection
criteria.
```

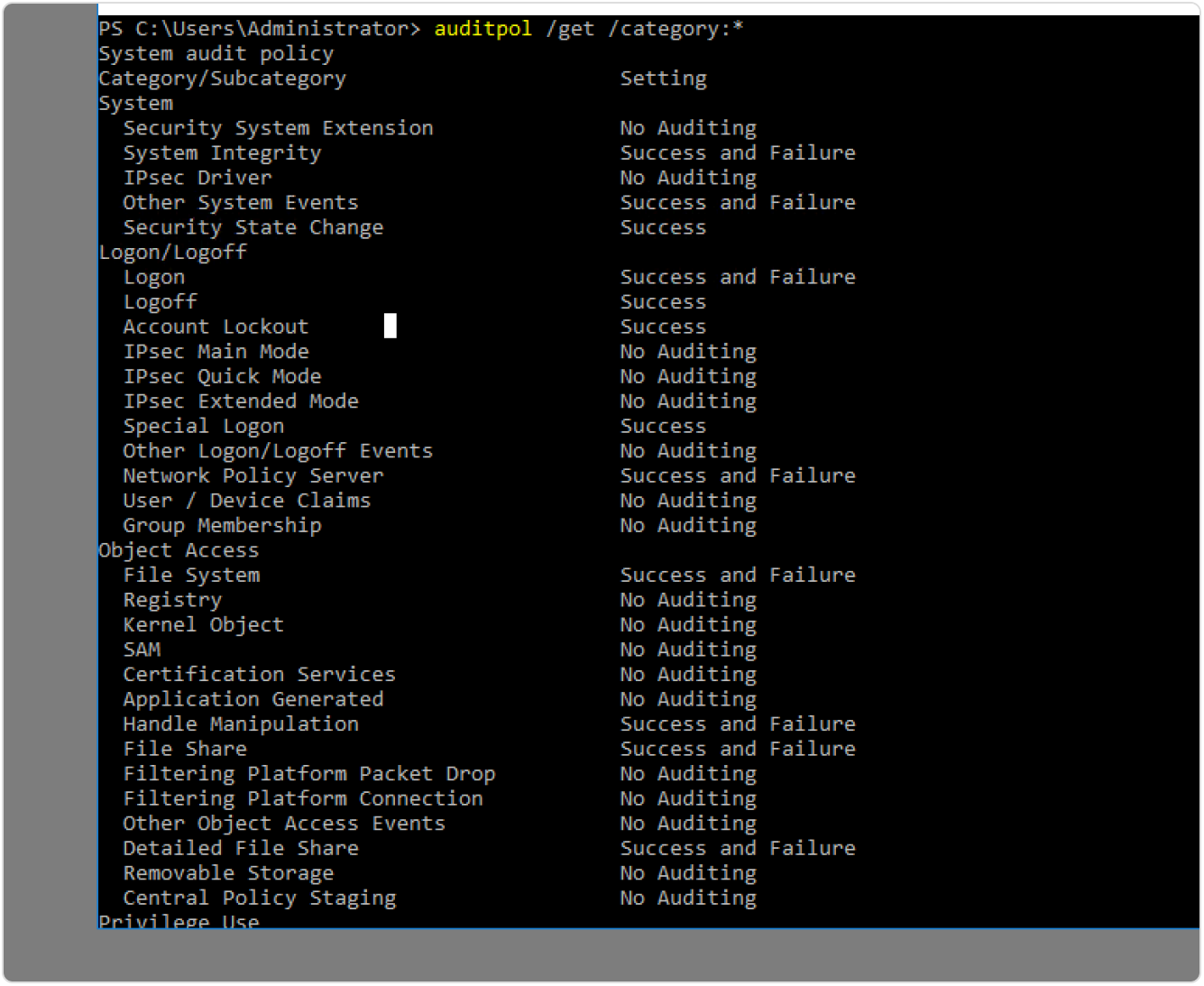
Lo mismo me pasó con el 4624 (inicio de sesión exitoso): cero resultados, a pesar de que el log de Seguridad tenía más de **97.000 eventos** acumulados. La primera sospecha fue que el filtro gráfico estuviera fallando, pero al confirmarlo también desde PowerShell quedó claro que el problema era real, no de interfaz.



Security log sin filtrar: 97.573 eventos, casi todos de tipo 4656 (acceso a objeto)

Para descartar que la auditoría estuviera simplemente desactivada, revisé la política completa:

```
auditpol /get /category:*
```



Logon y Special Logon en "Success" — la auditoría sí estaba activa

La auditoría de inicio de sesión estaba perfectamente activa. Entonces, ¿por qué no había ni un solo 4624/4672 entre 97.000 eventos? La respuesta estaba en las propiedades del log: tamaño máximo de **64 MB** con política **"Overwrite events as needed (oldest events first)"**, combinado con la auditoría de **File System (Object Access)** puesta en "Success and Failure" — que genera un volumen enorme de eventos por cada archivo tocado. Con un log tan pequeño y tanto ruido, Windows terminó sobrescribiendo solo los eventos raros de logon y privilegios (que databan de 2019) muchísimo antes de que yo entrara a investigar. Al no estar activado el modo "Archive the log when full", tampoco quedaban copias `.evtx` archivadas que recuperar.

Es una lección real de auditoría, no solo un contratiempo del ejercicio: tener la auditoría de File System en "Success and Failure" sin ajustar el tamaño del log puede literalmente **enterrar la evidencia que más importa** (logons, escaladas de privilegio) bajo ruido de bajo valor. Es

exactamente el tipo de hallazgo que un analista debería señalar en un informe real — "la configuración de logging impidió reconstruir el evento X" es una conclusión legítima, no un fallo de la investigación.

Para esa pregunta en concreto acabé verificando la hora (4:04:49 PM del 3/2/2019) contra la documentación pública de la sala, dejando constancia de que no la obtuve de mi propia instancia — pero el resto de la cronología (copia del backdoor a las 4:45 PM, primer disparo de la tarea a las 4:49 PM) sí la reconstruí directamente, sin depender de ese evento.

Conclusión

El atacante comprometió la máquina el 2 de marzo de 2019, escaló a privilegios especiales, se movió lateralmente con PsExec y dejó dos mecanismos de persistencia: una tarea programada camuflada como limpieza de sistema que relanza a diario un backdoor de Netcat en PowerShell, y una cuenta adicional (`Jenny`) metida en el grupo de Administradores. El hallazgo que más me llevé de este ejercicio no fue ninguna respuesta concreta de la sala, sino entender por qué el log de Seguridad se quedó mudo justo donde más falta hacía: una política de auditoría demasiado ruidosa sobre un log demasiado pequeño puede destruir la evidencia sin que nadie la borre a propósito.